Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07 Информационные системы и программирование

ПРАКТИЧЕСКАЯ РАБОТА № 4

Дисциплина(модуль)/предмет МДК 02.02. Инструментальные средства разработки и отладки программного обеспечения

Тема: Проектирование контрактов данных и валидация для подсистемы контроля доступа (СКУД)

Студент (ка)/учащийся: Иванов Егор Михайлович / _____ /
Ф. И. О. подпись

Группа / Класс И14-23-9

Руководитель: Чудинова А.А. / _____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «10» октября 2026 г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 КОНТРАКТЫ ДАННЫХ В ИНТЕГРАЦИОННОМ ШЛЮЗЕ СКУД.....4
2 ЗАДАНИЕ 1. JSON SCHEMA ДЛЯ СОБЫТИЙ ПРОХОДА.....5
2.1 Отображение требований на ключевые слова схемы.....5
2.2 Текст JSON Schema.....5
2.3 Тестирование схемы.....6
3 ЗАДАНИЕ 2. XSD-СХЕМА ДЛЯ ГОСТЕВЫХ ПРОПУСКОВ.....7
3.1 XML-документ заявки.....7
3.2 XSD-схема.....7
3.3 Проверка XML по XSD.....8
4 ЗАДАНИЕ 3. ДЕСЕРИАЛИЗАЦИЯ И ВАЛИДАЦИЯ НА C#.....9
4.1 Структура проекта.....9
4.2 Объектная модель и настройки десериализации.....9
4.3 Обработка ошибок и бизнес-правила.....10
4.4 Результаты выполнения.....11
ЗАКЛЮЧЕНИЕ.....13
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....14
ПРИЛОЖЕНИЕ А. ИСХОДНЫЙ КОД PROGRAM.CS.....15

ВВЕДЕНИЕ

Цель работы — закрепить навыки проектирования схем валидации данных (JSON Schema и XSD), а также реализации безопасной десериализации и проверки бизнес-правил на языке C# (.NET) на примере интеграции системы контроля и управления доступом (СКУД) [1].

Задачи работы:

– разработать JSON Schema `badge_event_schema.json` для событий прохода сотрудников по RFID-картам, запрещающую лишние поля, и проверить её на корректном и ошибочном событиях;

– разработать XML-документ заявки на гостевой пропуск и XSD-схему `visitor_pass.xsd` с ограничением минимальной длины имени посетителя;

– реализовать консольное приложение на C#, которое десериализует JSON-событие прохода в объектную модель с помощью System.Text.Json, перехватывает JsonException и проверяет бизнес-правило для номера точки прохода.

Все файлы работы размещены в каталоге `pr4-skud` репозитория `shop-integration`: схемы — в `schemas`, примеры сообщений — в `samples`, проект Visual Studio — в `AccessControl.sln`.

1 КОНТРАКТЫ ДАННЫХ В ИНТЕГРАЦИОННОМ ШЛЮЗЕ СКУД

Интеграционный шлюз корпоративной системы безопасности обрабатывает два потока данных: телеметрию турникетов (события прохода по RFID-картам в формате JSON) и заявки бюро пропусков на гостевые пропуска (документы XML) [1]. Каждый поток описывается контрактом данных — формальной схемой, которой обязаны соответствовать сообщения. Контракт выполняет три функции:

– документирует формат обмена для всех участников интеграции: разработчики считывателей и бюро пропусков знают, какие поля и в каком формате отправлять;

– позволяет автоматически отклонять неверные сообщения на входе в шлюз, до того как они попадут в бизнес-логику;

– защищает систему безопасности: запрет лишних полей и строгие типы закрывают путь для подмены данных и внедрения непредусмотренных атрибутов.

Для JSON используется стандарт JSON Schema (версия Draft 2020-12) [2], для XML — язык XML Schema Definition (XSD 1.0) [3]. Проверка схемой касается структуры и формата сообщения. Бизнес-правила, которые зависят от смысла данных, дополнительно проверяются в программном коде (задание 3).

2 ЗАДАНИЕ 1. JSON SCHEMA ДЛЯ СОБЫТИЙ ПРОХОДА

2.1 Отображение требований на ключевые слова схемы

Таблица 1 – Требования к событию прохода и их реализация в JSON Schema

Поле / требование	Требование задания	Ключевые слова JSON Schema
badge_id	Строка, шестнадцатеричный RFID-код из 8 символов	"type": "string", "pattern": "^[0-9A-Fa-f]{8}$"
checkpoint_id	Целое положительное число	"type": "integer", "minimum": 1
event_type	Только «ENTRY» или «EXIT»	"type": "string", "enum": ["ENTRY", "EXIT"]
timestamp	Дата и время по ISO 8601	"type": "string", "format": "date-time"
Все четыре поля	Обязательны (иначе событие теряет смысл)	"required": [...]
Лишние поля	Запрещены	"additionalProperties": false

Положительность номера точки прохода задана ограничением `minimum: 1`: для целых чисел это эквивалентно условию «больше нуля». Формат `date-time` соответствует профилю RFC 3339 [8] стандарта ISO 8601, то есть дата, время и часовой пояс обязательны (например, 2026-09-27T13:00:00Z).

2.2 Текст JSON Schema

Листинг 1 — файл `schemas/badge_event_schema.json`:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.local/skud/badge_event_schema.json",
  "title": "BadgeEvent",
  "description": "Событие прохода сотрудника через турникет (СКУД -> сервер
мониторинга)",
  "type": "object",
  "properties": {
    "badge_id": {
      "description": "Уникальный идентификатор RFID-карты: 8 шестнадцатеричных
символов",
      "type": "string",
      "pattern": "^[0-9A-Fa-f]{8}$"
    },
    "checkpoint_id": {
      "description": "Номер точки прохода (турникета), целое положительное число",
      "type": "integer",
      "minimum": 1
    },
    "event_type": {
      "description": "Направление прохода: ENTRY — вход, EXIT — выход",
      "type": "string",
      "enum": ["ENTRY", "EXIT"]
    },
    "timestamp": {
      "description": "Дата и время события по ISO 8601, например 2026-09-
27T13:00:00Z",
      "type": "string",
```

```
      "format": "date-time"
    }
  },
  "required": ["badge_id", "checkpoint_id", "event_type", "timestamp"],
  "additionalProperties": false
}
```

2.3 Тестирование схемы

Схема проверена скриптом `validate_schemas.py`, использующим библиотеку jsonschema с проверкой форматов [4]. Скрипт сначала проверяет корректность самой схемы по метасхеме Draft 2020-12, затем — тестовые события. В ошибочное событие специально внесено пять нарушений, по одному на каждое правило схемы.

Корректное событие (`samples/badge_event_valid.json`):

```
{"badge_id": "A1B2C3D4", "checkpoint_id": 3, "event_type": "ENTRY", "timestamp":
"2026-09-27T13:00:00Z"}
```

Ошибочное событие (`samples/badge_event_invalid.json`):

```
{"badge_id": "XYZ-0001", "checkpoint_id": 0, "event_type": "IN", "timestamp":
"27.09.2026 13:00", "door": "главный вход"}
```

Таблица 2 – Результат проверки событий по JSON Schema

Событие	Результат	Сообщения валидатора
badge_event_valid.json	VALID	—
badge_event_invalid.json	INVALID	(корень): Additional properties are not allowed ('door' was unexpected) badge_id: 'XYZ-0001' does not match '^[0-9A-Fa-f]{8}$' checkpoint_id: 0 is less than the minimum of 1 event_type: 'IN' is not one of ['ENTRY', 'EXIT'] timestamp: '27.09.2026 13:00' is not a 'date-time'

Каждое из пяти нарушений обнаружено своим правилом, корректное событие принято. Схема работает так, как требует задание.

3 ЗАДАНИЕ 2. XSD-СХЕМА ДЛЯ ГОСТЕВЫХ ПРОПУСКОВ

3.1 XML-документ заявки

Заявка на гостевой пропуск имеет корневой элемент `<VisitorPassRequest>` и четыре дочерних элемента. Листинг 2 — файл `samples/visitor_pass.xml`:

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Заявка на гостевой пропуск: проходит проверку по visitor_pass.xsd -->
<VisitorPassRequest xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
                    xsi:noNamespaceSchemaLocation="../schemas/visitor_pass.xsd">
  <PassId>GP-2026-000417</PassId>
  <VisitorName>Смирнова Анна Викторовна</VisitorName>
  <HostEmployeeId>10582</HostEmployeeId>
  <ValidUntil>2026-09-27T18:00:00+03:00</ValidUntil>
</VisitorPassRequest>
```

3.2 XSD-схема

Таблица 3 – Требования к заявке и их реализация в XSD

Элемент	Требование	Реализация в XSD
PassId	Строковый идентификатор заявки	простой тип PassIdType: xs:string, minLength 1, maxLength 32
VisitorName	ФИО посетителя, обязательная строка, не короче 2 символов	простой тип VisitorNameType: xs:string, minLength 2, maxLength 150, whiteSpace collapse
HostEmployeeId	Табельный номер принимающего сотрудника, целое число	xs:int
ValidUntil	Дата и время окончания действия пропуска	xs:dateTime
Все элементы	Обязательны и идут в заданном порядке	xs:sequence; minOccurs по умолчанию 1

Ограничение `whiteSpace collapse` сжимает пробелы перед проверкой длины, поэтому строка из пробелов не пройдёт как «имя» длиной 2 символа. Листинг 3 — файл `schemas/visitor_pass.xsd`:

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- XSD-схема заявки на гостевой пропуск (бюро пропусков -> шлюз СКУД) -->
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
elementFormDefault="qualified">

  <!-- ФИО посетителя: обязательная строка не короче 2 символов -->
  <xs:simpleType name="VisitorNameType">
    <xs:restriction base="xs:string">
      <xs:minLength value="2"/>
      <xs:maxLength value="150"/>
      <xs:whiteSpace value="collapse"/>
    </xs:restriction>
  </xs:simpleType>

  <!-- Идентификатор заявки: непустая строка -->
  <xs:simpleType name="PassIdType">
    <xs:restriction base="xs:string">
```

```
      <xs:minLength value="1"/>
      <xs:maxLength value="32"/>
    </xs:restriction>
  </xs:simpleType>

  <xs:element name="VisitorPassRequest">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="PassId" type="PassIdType"/>
        <xs:element name="VisitorName" type="VisitorNameType"/>
        <xs:element name="HostEmployeeId" type="xs:int"/>
        <xs:element name="ValidUntil" type="xs:dateTime"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

3.3 Проверка XML по XSD

XSD-схема проверена двумя независимыми средствами: библиотекой lxml в скрипте `validate_schemas.py` и встроенным классом .NET XmlReader с настройкой ValidationType.Schema в консольном приложении (раздел 4). Ошибочная заявка `samples/visitor_pass_invalid.xml` содержит имя из одного символа, табельный номер «A-105» и дату без времени.

Таблица 4 – Результат проверки заявок по XSD

Заявка	Результат	Ошибки (lxml)
visitor_pass.xml	VALID	—
visitor_pass_invalid.xml	INVALID	строка 5: VisitorName — длина 1 меньше minLength 2 строка 6: HostEmployeeId — 'A-105' не является xs:int строка 7: ValidUntil — '2026-09-27' не является xs:dateTime

4 ЗАДАНИЕ 3. ДЕСЕРИАЛИЗАЦИЯ И ВАЛИДАЦИЯ НА C#

4.1 Структура проекта

Консольное приложение создано на платформе .NET 8 и не использует сторонних пакетов: для JSON применяется System.Text.Json [5], для XSD — System.Xml.Schema [7]. Решение `AccessControl.sln` открывается в Visual Studio, проект `AccessControl` содержит один файл исходного кода `Program.cs`. Файлы схем и примеров копируются в каталог сборки рядом с исполняемым файлом, поэтому программа находит их при запуске из Visual Studio.

4.2 Объектная модель и настройки десериализации

За основу взят шаблон кода из задания. При его анализе выявлены и исправлены два недочёта:

– в шаблоне после условия `if (badgeEvent.CheckpointId <= 0)` стоит лишний символ «5» — код с ним не компилируется;

– тестовая строка шаблона смешивает стили имён (badgeId, checkpoint_id, eventType). Настройка PropertyNameCaseInsensitive игнорирует только регистр букв, но не подчёркивания: «checkpoint_id» не сопоставляется со свойством CheckpointId. Поле молча остаётся равным 0, и корректное событие ошибочно вызывает тревогу.

Чтобы модель соответствовала контракту из задания 1, имена JSON в формате snake_case заданы явно атрибутами JsonPropertyName:

```
public record BadgeEventDto(
    [property: JsonPropertyName("badge_id")] string? BadgeId,
    [property: JsonPropertyName("checkpoint_id")] int CheckpointId,
    [property: JsonPropertyName("event_type")] string? EventType,
    [property: JsonPropertyName("timestamp")] DateTime Timestamp
);
```

Настройки сериализатора:

```
private static readonly JsonSerializerOptions Options = new()
{
    PropertyNameCaseInsensitive = true,
    UnmappedMemberHandling = JsonUnmappedMemberHandling.Disallow
};
```

PropertyNameCaseInsensitive = true установлено по требованию задания: «BADGE_ID» и «badge_id» считаются одним полем. Настройка UnmappedMemberHandling.Disallow (доступна с .NET 8) [6] — программный аналог `additionalProperties: false`: сообщение с неизвестным полем вызывает JsonException, а не обрабатывается с потерей этого поля.

4.3 Обработка ошибок и бизнес-правила

Метод `ProcessBadgeScan` выполняет десериализацию в блоке try-catch с перехватом JsonException. Это исключение возникает при битом синтаксисе, несовпадении типа (например, строка вместо числа) и лишнем поле; программа выводит сообщение с деталями и продолжает работу, а не аварийно завершается. После успешной десериализации проверки выполняются по порядку:

1) результат null (сообщение «null») — «[Ошибка]: Пустой пакет данных от считывателя»;

2) checkpoint_id ≤ 0 — предупреждение системы безопасности «[Тревога]: Некорректный ID точки прохода» (обязательное правило задания);

3) badge_id не соответствует шаблону ^[0-9A-Fa-f]{8}$ — тревога о недопустимой карте;

4) event_type не равен ENTRY или EXIT — ошибка типа события;

5) timestamp не задан — ошибка отсутствия времени;

6) иначе — «[Доступ разрешен]» с номером карты, точкой прохода, направлением и временем.

Проверки 3–5 дублируют правила JSON Schema. Это сделано намеренно: шлюз не должен полагаться на то, что сообщение уже было проверено схемой где-то раньше (принцип «не доверяй входным данным»).

4.4 Результаты выполнения

При запуске без аргументов программа обрабатывает набор из 11 сообщений — корректных и ошибочных — и затем проверяет обе XML-заявки по XSD. Результаты приведены в таблице 5.

Таблица 5 – Результаты обработки тестовых сообщений

№	Сообщение	Ожидаемая реакция	Фактический вывод программы
1	Корректное событие	Доступ разрешён	[Доступ разрешен]: Карта A1B2C3D4 зафиксирована на точке 3 (ENTRY)
2	Имена полей в другом регистре	Доступ разрешён	[Доступ разрешен]: Карта 0F0F0F0F … на точке 12 (EXIT)
3	checkpoint_id = 0	Тревога	[Тревога]: Некорректный ID точки прохода #0!
4	checkpoint_id = -7	Тревога	[Тревога]: Некорректный ID точки прохода #-7!
5	Нет закрывающей скобки	JsonException	[Критическая ошибка десериализации]: … open JSON object … should be closed
6	checkpoint_id строкой «три»	JsonException	[Критическая ошибка десериализации]: … could not be converted … Path: $.checkpoint_id
7	Лишнее поле door	JsonException	[Критическая ошибка десериализации]: … 'door' could not be mapped …
8	badge_id = XYZ-0001	Тревога	[Тревога]: Недопустимый идентификатор карты 'XYZ-0001'
9	event_type = IN	Ошибка	[Ошибка]: Неизвестный тип события 'IN'
10	null	Ошибка	[Ошибка]: Пустой пакет данных от считывателя.
11	Строка из шаблона задания	JsonException	[Критическая ошибка десериализации]: … 'badgeId' could not be mapped …
12	visitor_pass.xml	Соответствует XSD	[XSD]: visitor_pass.xml — заявка соответствует схеме.
13	visitor_pass_invalid.xml	3 ошибки	[XSD]: visitor_pass_invalid.xml — найдено ошибок: 3.

Фактическая реакция программы во всех 13 случаях совпала с ожидаемой. Снимок экрана выполнения консольного приложения приведён на рисунке 1.

Рисунок 1 – Результат выполнения консольного приложения AccessControl: обработка корректных и ошибочных запросов (лист повёрнут)

ЗАКЛЮЧЕНИЕ

В ходе практической работы спроектированы контракты данных для двух потоков интеграционного шлюза СКУД. JSON Schema событий прохода задаёт тип, формат и обязательность всех четырёх полей и запрещает лишние поля; проверка показала, что корректное событие принимается, а все пять нарушений в ошибочном событии обнаруживаются. XSD-схема гостевых пропусков ограничивает минимальную длину имени посетителя двумя символами и задаёт типы xs:int и xs:dateTime; ошибочная заявка отклоняется с указанием трёх ошибок.

Разработано консольное приложение на C# (.NET 8), которое десериализует JSON-событие с помощью System.Text.Json с настройкой PropertyNameCaseInsensitive, перехватывает JsonException при повреждённом сообщении и выдаёт предупреждение системы безопасности при нулевом или отрицательном номере точки прохода. В шаблоне кода выявлены и исправлены синтаксическая ошибка и несоответствие имён полей контракту. На 13 тестовых сообщениях приложение отработало в соответствии с ожиданиями.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Практическая работа № 4 «Проектирование контрактов данных и валидация для подсистемы контроля доступа (СКУД)» : задание по МДК 02.02 / СПбИЭУ. – Санкт-Петербург, 2026. – Текст : электронный // ЭИОС СПбИЭУ. – URL: https://spbiem.tech (дата обращения: 10.10.2026).

2. JSON Schema: A Media Type for Describing JSON Documents. Draft 2020-12. – URL: https://json-schema.org/draft/2020-12/json-schema-core (дата обращения: 10.10.2026). – Текст : электронный.

3. W3C XML Schema Definition Language (XSD) 1.1 Part 2: Datatypes. – URL: https://www.w3.org/TR/xmlschema11-2/ (дата обращения: 10.10.2026). – Текст : электронный.

4. jsonschema — An implementation of JSON Schema for Python : документация. – URL: https://python-jsonschema.readthedocs.io/ (дата обращения: 10.10.2026). – Текст : электронный.

5. Microsoft Learn. Сериализация и десериализация JSON в .NET (System.Text.Json). – URL: https://learn.microsoft.com/ru-ru/dotnet/standard/serialization/system-text-json/overview (дата обращения: 10.10.2026). – Текст : электронный.

6. Microsoft Learn. Обработка элементов, отсутствующих в модели (UnmappedMemberHandling). – URL: https://learn.microsoft.com/ru-ru/dotnet/standard/serialization/system-text-json/missing-members (дата обращения: 10.10.2026). – Текст : электронный.

7. Microsoft Learn. Проверка XML-документа по схеме XSD с помощью XmlReader. – URL: https://learn.microsoft.com/ru-ru/dotnet/standard/data/xml/xml-schema-xsd-validation-with-xmlschemaset (дата обращения: 10.10.2026). – Текст : электронный.

8. RFC 3339. Date and Time on the Internet: Timestamps. – IETF, 2002. – URL: https://www.rfc-editor.org/rfc/rfc3339 (дата обращения: 10.10.2026). – Текст : электронный.

ПРИЛОЖЕНИЕ А. ИСХОДНЫЙ КОД PROGRAM.CS

```
// ПР №4. Контракты данных и валидация для подсистемы контроля доступа (СКУД).
// Задание 3: десериализация JSON-события прохода и проверка бизнес-правил.
// Дополнительно: проверка XML-заявки на гостевой пропуск по XSD (задание 2).
using System;
using System.IO;
using System.Text;
using System.Text.Json;
using System.Text.Json.Serialization;
using System.Text.RegularExpressions;
using System.Xml;
using System.Xml.Schema;

namespace AccessControl;

// Объектная модель события. Имена в JSON — snake_case, как в badge_event_schema.json,
// поэтому каждому свойству явно задано имя JSON через JsonPropertyName.
public record BadgeEventDto(
    [property: JsonPropertyName("badge_id")] string? BadgeId,
    [property: JsonPropertyName("checkpoint_id")] int CheckpointId,
    [property: JsonPropertyName("event_type")] string? EventType,
    [property: JsonPropertyName("timestamp")] DateTime Timestamp
);

public static class AccessControlHandler
{
    private static readonly Regex BadgePattern = new("^[0-9A-Fa-f]{8}$");

    private static readonly JsonSerializerOptions Options = new()
    {
        PropertyNameCaseInsensitive = true,                          // требование задания
        UnmappedMemberHandling = JsonUnmappedMemberHandling.Disallow // лишние поля -> ошибка
(как additionalProperties: false)
    };

    public static void ProcessBadgeScan(string rawJson)
    {
        try
        {
            BadgeEventDto? badgeEvent = JsonSerializer.Deserialize<BadgeEventDto>(rawJson,
Options);

            if (badgeEvent == null)
            {
                Print(ConsoleColor.Yellow, "[Ошибка]: Пустой пакет данных от считывателя.");
                return;
            }

            if (badgeEvent.CheckpointId <= 0)
            {
                Print(ConsoleColor.Red, $"[Тревога]: Некорректный ID точки прохода
#{badgeEvent.CheckpointId}!");
                return;
            }

            if (badgeEvent.BadgeId == null || !BadgePattern.IsMatch(badgeEvent.BadgeId))
            {
                Print(ConsoleColor.Red, $"[Тревога]: Недопустимый идентификатор карты
'{badgeEvent.BadgeId}' (нужно 8 hex-символов).");
                return;
            }

            if (badgeEvent.EventType is not ("ENTRY" or "EXIT"))
            {
                Print(ConsoleColor.Yellow, $"[Ошибка]: Неизвестный тип события
'{badgeEvent.EventType}' (допустимо ENTRY или EXIT).");
                return;
            }

            if (badgeEvent.Timestamp == default)
            {
                Print(ConsoleColor.Yellow, "[Ошибка]: В событии нет времени прохода
(timestamp).");
                return;
            }

            Print(ConsoleColor.Green,
```

```
                $"[Доступ разрешен]: Карта {badgeEvent.BadgeId} зафиксирована на точке
{badgeEvent.CheckpointId} " +
                $"({badgeEvent.EventType}) в {badgeEvent.Timestamp:yyyy-MM-dd HH:mm:ss} UTC");
        }
        catch (JsonException ex)
        {
            Print(ConsoleColor.Magenta, $"[Критическая ошибка десериализации]: Поврежден
формат сообщения. Детали: {ex.Message}");
        }
    }

    // Проверка XML-заявки на гостевой пропуск по XSD-схеме средствами .NET
    public static void ValidateVisitorPass(string xmlPath, string xsdPath)
    {
        var settings = new XmlReaderSettings { ValidationType = ValidationType.Schema };
        settings.Schemas.Add(null, xsdPath);
        int errors = 0;
        settings.ValidationEventHandler += (_, e) =>
        {
            errors++;
            Print(ConsoleColor.Red, $"    строка {e.Exception.LineNumber}: {e.Message}");
        };
        using (var reader = XmlReader.Create(xmlPath, settings))
        {
            while (reader.Read()) { }
        }
        if (errors == 0)
            Print(ConsoleColor.Green, $"[XSD]: {Path.GetFileName(xmlPath)} — заявка
соответствует схеме.");
        else
            Print(ConsoleColor.Yellow, $"[XSD]: {Path.GetFileName(xmlPath)} — найдено ошибок:
{errors}.");
    }

    private static void Print(ConsoleColor color, string text)
    {
        Console.ForegroundColor = color;
        Console.WriteLine(text);
        Console.ResetColor();
    }

    public static void Main(string[] args)
    {
        Console.OutputEncoding = Encoding.UTF8;
        string baseDir = AppContext.BaseDirectory;

        // Режим 1: путь к JSON-файлу в аргументах -> обработать только его
        if (args.Length > 0)
        {
            foreach (var path in args)
            {
                Console.WriteLine($"> Файл: {path}");
                ProcessBadgeScan(File.ReadAllText(path));
            }
            return;
        }

        // Режим 2 (по умолчанию): демонстрация на корректных и ошибочных запросах
        var tests = new (string Title, string Json)[]
        {
            ("1. Корректное событие",
             "{\"badge_id\": \"A1B2C3D4\", \"checkpoint_id\":
3, \"event_type\": \"ENTRY\", \"timestamp\": \"2026-09-27T13:00:00Z\"}"),
            ("2. Регистр имён полей другой (PropertyNameCaseInsensitive)",
             "{\"BADGE_ID\": \"0F0F0F0F\", \"Checkpoint_Id\":
12, \"Event_Type\": \"EXIT\", \"TimeStamp\": \"2026-09-27T18:05:00Z\"}"),
            ("3. checkpoint_id = 0",
             "{\"badge_id\": \"A1B2C3D4\", \"checkpoint_id\":
0, \"event_type\": \"ENTRY\", \"timestamp\": \"2026-09-27T13:00:00Z\"}"),
            ("4. checkpoint_id отрицательный",
             "{\"badge_id\": \"A1B2C3D4\", \"checkpoint_id\": -
7, \"event_type\": \"EXIT\", \"timestamp\": \"2026-09-27T13:00:00Z\"}"),
            ("5. Битый синтаксис JSON (нет закрывающей скобки)",
             "{\"badge_id\": \"A1B2C3D4\", \"checkpoint_id\": 3, \"event_type\": \"ENTRY\""),
            ("6. Неверный тип: checkpoint_id строкой",

"{\"badge_id\": \"A1B2C3D4\", \"checkpoint_id\": \"три\", \"event_type\": \"ENTRY\", \"timesta
mp\": \"2026-09-27T13:00:00Z\"}"),
            ("7. Лишнее поле в сообщении",
```

```
             "{\"badge_id\": \"A1B2C3D4\", \"checkpoint_id\":
3, \"event_type\": \"ENTRY\", \"timestamp\": \"2026-09-27T13:00:00Z\", \"door\": \"main\"}"),
            ("8. Неверный формат карты",
             "{\"badge_id\": \"XYZ-0001\", \"checkpoint_id\":
3, \"event_type\": \"ENTRY\", \"timestamp\": \"2026-09-27T13:00:00Z\"}"),
            ("9. Неизвестный тип события",
             "{\"badge_id\": \"A1B2C3D4\", \"checkpoint_id\":
3, \"event_type\": \"IN\", \"timestamp\": \"2026-09-27T13:00:00Z\"}"),
            ("10. Пустой пакет (null)", "null"),
            ("11. Строка из шаблона задания (смешанные стили имён)",
             "{\"badgeId\": \"A1B2C3D4\", \"checkpoint_id\":
3, \"eventType\": \"ENTRY\", \"timestamp\": \"2026-09-27T13:00:00Z\"}"),
        };

        Console.WriteLine("СКУД: обработка событий прохода (JSON)");
        foreach (var (title, json) in tests)
        {
            Console.WriteLine();
            Console.WriteLine(title);
            Console.WriteLine($"    {json}");
            ProcessBadgeScan(json);
        }

        Console.WriteLine();
        Console.WriteLine("Бюро пропусков: проверка заявок (XML по XSD)");
        string xsd = Path.Combine(baseDir, "schemas", "visitor_pass.xsd");
        ValidateVisitorPass(Path.Combine(baseDir, "samples", "visitor_pass.xml"), xsd);
        ValidateVisitorPass(Path.Combine(baseDir, "samples", "visitor_pass_invalid.xml"),
xsd);
    }
}
```